

Asignatura: Desarrollo de Aplicaciones Móviles Nativas.
Tema: Operadores y estructuras de control de Kotlin.

Operadores algebraicos:

Aritméticos	<code>+, -, *, \, %</code>
Asignación	<code>=, +=, -=, *=, /=, %</code>
Comparación	<code>==, !=, >, <, >=, <=</code>
Lógicos	<code>&&</code> es AND, <code> </code> es OR, <code>!</code> es NOT
Incremento y Decremento	<code>++, --</code>
Comparación referencial	<code>===, !==</code>

Operadores de seguridad Null (Null Safety):

Android utiliza `ViewBinding` o `Compose` para la inyección de dependencias y vistas.

?	Llamada segura. Invoca a un método o propiedad solamente si el objeto no es nulo. Ejemplo: <code>view?.visibility = View.VISIBLE</code>
!!	Operador de assertion no nulo. Fuerza la ejecución asumiendo que el valor no es nulo. <i>Usarse con extrema precaución.</i>
?:	Operador Elvis. Devuelve un valor predeterminado si la expresión de la izquierda resulta ser nula. Ejemplo: <code>val texto = nombreUsuario ?: "Invitado"</code>

Operadores de rangos y colecciones:

..	Crea un rango de valores. Ejemplo: <code>for(i in 1..5)</code>
in !in	Verifica si un elemento pertenece o no a una colección o rango. Ejemplo: <code>if(numero in 1..10)</code>

Operadores de conversión y tipos:

as as?	(Casting): Convierte un objeto a otro tipo. <code>as?</code> realiza una conversión segura y devuelve <code>null</code> si falla en lugar de lanzar una excepción
is !is	(Verifica el tipo): Comprueba si un objeto es de un tipo determinado. Ejemplo: <code>if(dato is String) { ... }</code>

Estructuras de Decisión:

if - else

En Kotlin, la estructura `if-else` también puede actuar como una expresión y podría devolver un valor. Utilizar el código con precaución.

```
val edad = 18
val esMayor = if (edad >= 18) {
    true
} else {
    false
}
```

when

Funciona similar al `switch` en otros lenguajes. No necesita `break`. Puede evaluar múltiples casos, comprobar rangos o evaluar tipos de datos.

```
val nota = 8
val evaluacion = when (nota) {
    in 0..4 -> "Suspenso"
    in 5..6 -> "Suficiente"
    in 7..8 -> "Notable"
    9, 10 -> "Sobresaliente"
    else -> "Calificación inválida"
}
```

Estructuras de Iteración:

for

Itera sobre cualquier elemento que proporcione un iterador (por ejemplo: listas, arrays o rangos).

```
// Iteración sobre un rango
for (i in 1..5) {
    println(i)
}

// Iteración sobre una lista
val lenguajes = listOf("Java", "Kotlin", "Python")
for (lenguaje in lenguajes) {          // Utilizar el código con precaución.

    println(lenguaje)
}
```

while y do-while

Ejecutan un bloque de código repetidamente si una condición es verdadera y do-while ejecuta al menos una vez el bloque antes de comprobar la condición.

```
var contador = 1
while (contador <= 5) {
    println(contador)
    contador++
}

do {
    println("Se ejecuta al menos una vez")
} while (false)          // Utilizar el código con precaución.
```

Expresiones de Salto (Jump Expressions)

Palabras clave que modifican el flujo de los ciclos:

return	Devuelve el valor desde la función más externa o desde una función anónima.
break	Termina la ejecución de un ciclo.
continue	Salta la iteración actual del ciclo y continúa con la siguiente.

DESARROLLO

Ejemplo. Abrir el IDE de Android Studio. Seleccionar un nuevo proyecto y la opción Empty Views Activity. Seleccionar el lenguaje Kotlin.

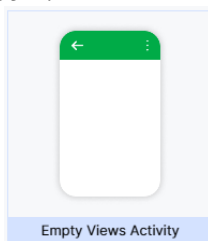


Figura 1. Seleccionar una actividad.

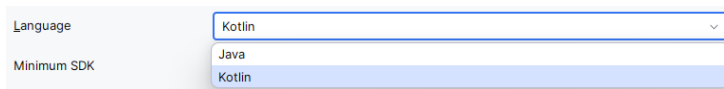


Figura 2. Seleccionar Kotlin.

1. Abrir el archivo MainActivity.kt, copiar y pegar el código siguiente para actualizar el contenido del editor.

```
import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
    override fun onCreate(b: Bundle?) {
        super.onCreate(b)
        setContentView(R.layout.activity_main)
        val xnatural = findViewById<EditText>(R.id.xnatural)
        val xboton = findViewById<Button>(R.id.xboton)
        val resultText = findViewById<TextView>(R.id.xcalcular)
```

```

xboton.setOnClickListener {
    val input = xnatural.text.toString()
    if (input.isNotEmpty()) {
        val numero = input.toInt()
        resultText.text = if (esPrimo(numero)) {
            "$numero es un número primo."
        } else {
            "$numero no es un número primo."
        }
    } else {
        resultText.text = "Por favor, ingresa un número."
    }
}
}

private fun esPrimo(num: Int): Boolean {
    if (num <= 1) return false
    for (i in 2..Math.sqrt(num.toDouble()).toInt()) {
        if (num % i == 0) return false
    }
    return true
}
}

```

2. Abrir el archivo `activity_main.xml`, copiar y pegar el código siguiente para actualizar el contenido del editor.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
    <EditText
        android:id="@+id/xnatural"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Ingresa un número natural"
        android:inputType="number"
        tools:ignore="MissingConstraints" />
    <TextView
        android:id="@+id/xcalcular"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        tools:ignore="MissingConstraints" />
    <Button
        android:id="@+id/xboton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Verificar"
        tools:ignore="MissingConstraints" />
</androidx.constraintlayout.widget.ConstraintLayout>

```



Figura 3. Ejecución de la aplicación.

3. Ejecutar la aplicación y probar con varios valores para verificar la operación.

Ejercicio 1. Completar el ejemplo anterior para verificar si el número es Fibonacci y Número Maravilloso.

Ejercicio 2. Incluir en el ejercicio 1 si el número, si es de tres o cuatro dígitos, se corresponde con la constante de Kaprekar.

NOTA. Generar el reporte `AlumnoKotlinNumerosGrupo.doc` y enviarlo al sitio indicado por el profesor.